

دانشگاه تهران

پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر

تمرین شماره: ۳

شبکه‌های عصبی و یادگیری عمیق

نام و نام خانوادگی: آرتین توسلی

شماره دانشجویی: ۸۱۰۱۰۲۵۴۳

نیم‌سال دوم

سال تحصیلی ۴۰۴-۴۰۵

فهرست مطالب

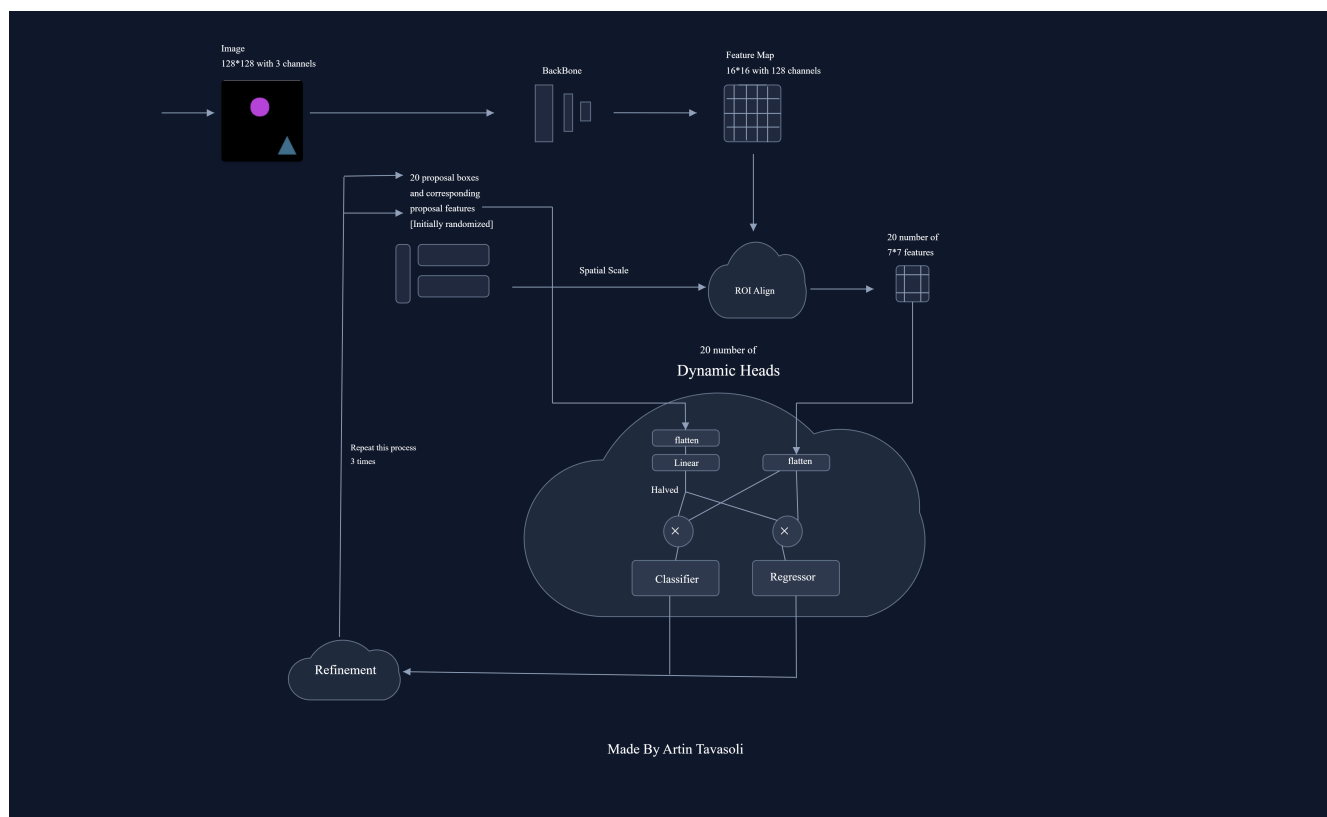
۳	Object Detection Using Sparse R-CNN	۱
۵	تئوری	۱.۱
۵	مدل‌های دو مرحله‌ای	۱.۱.۱
۶	مقایسه Faster R-CNN و Fasd R-CNN	۲.۱.۱
۷	معماری Faster R-CNN	۳.۱.۱
۹	معماری Sparse R-CNN	۴.۱.۱
۱۰	پیاده‌سازی	۲.۱
۱۰	آماده‌سازی داده	۱.۲.۱
۱۱	تقسیم‌بندی داده‌ها و ساخت dataloader	۲.۲.۱
۱۲	پیاده‌سازی معماری Sparse R-CNN	۳.۲.۱
۱۳	تعریف تابع هزینه و تطبیق دوطرفه	۴.۲.۱
۱۵	آموزش مدل	۵.۲.۱
۱۷	ارزیابی نهایی و تحلیل بصری	۶.۲.۱

فهرست تصاویر

۴		۱.۱ معماری مدل Sparse R-CNN
۸		۲.۱ تصویری معماری Faster R-CNN (از medium.com برداشته شده است).
۱۱		۳.۱ پنج نمونه تصادفی از مجموعه داده
۱۶		۴.۱ نمودار آموزش و تست مدل.
۱۷		۵.۱ پیش‌بینی‌های مدل روی ۱۰ نمونه از داده‌های تست.
۱۸		۶.۱ روند تکامل ۲۰ جعبه‌ی پیشنهادی

سوال ۱

Object Detection Using Sparse R-CNN



شکل ۱.۱: معماری مدل Sparse R-CNN

۱.۱ تئوری

۱.۱.۱ مدل‌های دو مرحله‌ای

چند مورد معماری دو مرحله‌ای

- معماری R-CNN (Region-based CNN)
- معماری SPP-Net (Spatial Pyramid Pooling Network)
- معماری Fast R-CNN
- معماری Faster R-CNN
- معماری FPN (Feature Pyramid Network)

تفاوت‌های اصلی معماری‌های دو مرحله‌ای با یک مرحله‌ای

نحوه عملکرد شبکه‌های دو مرحله‌ای:

در مرحله اول، هدف فقط پیدا کردن مناطقی است که احتمالاً حاوی یک شیء مهم هستند یعنی Region Proposal ها. مدل کل تصویر را اسکن می‌کند و صدها هزار کاندیدای مختلف را بررسی می‌کند تا در نهایت مجموعه کوچکتري از جعبه‌ها را استخراج کند. در این مرحله اهمیت نمیدهیم داخل جعبه چه شی‌ای قرار گرفته است (مثلاً دایره هست یا مثلث یا مربع) و فقط اهمیت میدهم مکانی که یک چیزی آنجا هست رو پیدا کنیم. الگوریتم‌هایی مثل Selective Search (در R-CNN) یا شبکه‌ی RPN (در Faster R-CNN) نقش اصلی را در این مرحله بازی می‌کنند.

در مرحله دوم، هدف refine و classification کردن مختصات جعبه‌ها میباشد. یعنی جعبه‌هایی که از مرحله پیش استخراج کردیم، ویژگی‌هایش استخراج میشود و بعد طبقه بندی میکنیم (داخل هر جعبه چه چیزی قرار دارد مثلاً دایره) و بعد رگرسیون هم داریم که مختصات جعبه را بهتر کند (مرزی که دور شی میگیرد فیت تر بشود).

نحوه عملکرد شبکه‌های یک مرحله‌ای:

کلاً مرحله‌ی Region Proposal را دور می‌زنند و تلاش می‌کنند همه چیز را در یک حرکت سریع و سراسر پیش‌بینی کنند. تصویر را به یک Grid متراکم تقسیم می‌کنند و Anchor Boxes با ابعاد، مقیاس‌ها و نسبت‌های مختلف را روی تمام این موقعیت‌های مکانی قرار می‌دهند. سپس، به صورت یک‌باره (Single-shot) و مستقیماً پیش‌بینی می‌کنند که داخل هر کدام از این جعبه‌های متراکم چه کلاسی قرار دارد و مختصات دقیق جعبه چقدر باید تغییر کند.

تفاوت اصلی در پردازش: شبکه‌های دو مرحله‌ای مسئله تشخیص را به دو گام ”مکان‌یابی تقریبی/فیلتر پس‌زمینه” و ”طبقه‌بندی دقیق/اصلاح مختصات” تقسیم می‌کنند. اما شبکه‌های یک مرحله‌ای کل تصویر را به عنوان یک Grid پیوسته می‌بینند و تشخیص را به عنوان یک مسئله رگرسیون تکی برای پیش‌بینی همزمان کلاس و مختصات در تمام خانه‌های گرید حل می‌کنند.

مزایا و معایب شبکه‌های دو مرحله‌ای نسبت به یک مرحله‌ای

مزیت: دقت بالاتر در تشخیص اشیاء (مخصوصاً وقتی اشیاء کوچیکن یا روی هم افتاده اند) (مثال‌های سخت حساب میشوند اینا) دقت بهتری نشون میدن تا (یک مرحله‌ای)

دلیل:

- فرض کنید در یک عکس فقط ۵ دایره وجود دارد. یک شبکه تک مرحله‌ای، روی کل عکس حدود ۱۰۰,۰۰۰ Anchor میندازد. از این تعداد، فقط چند تاش برای دایره هاست و تعداد خیلی زیادیش برای پس زمینه هست (شی کوچیکتر یعنی نویز پس زمینه بیشتر هم هست). این حجم عظیم از پس زمینه باعث می‌شود شبکه در طول آموزش به اصطلاح غرق شود و سیگنال‌های مربوط به اشیاء واقعی در میان نویز پس زمینه گم شوند. در مدل‌های دو مرحله‌ای، مرحله اول (RPN) نقش یک فیلتر قدرتمند را بازی می‌کند. این مرحله ده‌ها هزار کادر بی‌ارزش را دور می‌ریزد و فقط حدود ۲۰۰۰ کادر مفید را به مرحله دوم می‌فرستد. در نتیجه، شبکه طبقه‌بند اصلی با یک داده‌ی بسیار تمیزتر و متعادل‌تر مواجه می‌شود و راحت‌تر یاد می‌گیرد.
- مدل‌های دو مرحله‌ای مکانیزمی به نام RoI Pooling (یا RoI Align) دارند. این عملیات دقیقاً همان بخشی از Feature Map که مربوط به یک کادر خاص است را بُرش می‌زند، استخراج می‌کند و به یک اندازه ثابت درمی‌آورد. این یعنی طبقه‌بند مرحله دوم، بدون هیچ‌گونه حواس‌پرتی یا نویز از پیکسل‌های اطراف، فقط و فقط به ماهیت همان شیء نگاه می‌کند. در مدل‌های یک مرحله‌ای این بُرش اختصاصی وجود ندارد.

عیب: سرعت پایین‌تر

در شبکه‌های یک مرحله‌ای، کل عکس یک باره و کاملاً موازی روی GPU پردازش می‌شود. اما در دو مرحله‌ای‌ها، وقتی در مرحله اول ۲۰۰۰ کادر انتخاب شد، شبکه باید ویژگی‌های این ۲۰۰۰ کادر را جداگانه (یا در دسته‌های کوچک) از شبکه‌های Fully Connected عبور دهد. این کار پردازش موازی و یکپارچه را می‌شکند و یک گلوگاه محاسباتی ایجاد می‌کند. (به جز این کلا دو مرحله ای pipeline طولانی تری دارد و همینطور RoI Pooling هم زمان میبرد ولی در یک مرحله ای اینا نیست.)

۲.۱.۱ مقایسه Faster R-CNN و Fast R-CNN

معماری Fast R-CNN:

ورودی: خود تصویر اصلی و لیستی شامل حدود ۲۰۰۰ Proposals که توسط یک الگوریتم مجزا و خارجی (مثلاً Selective Search) تولید شده‌اند. شبکه پایه استخراج ویژگی (Backbone): تصویر وارد یک شبکه عصبی پیچشی عمیق می‌شود. برخلاف R-CNN که تصویر را ۲۰۰۰ بار خرد می‌کرد و به شبکه می‌داد، اینجا تصویر فقط یک بار پردازش می‌شود. خروجی این مرحله، یک Feature Map از کل تصویر است. لایه RoI Pooling (مغز متفکر Fast R-CNN): حالا ما یک نقشه Feature Map داریم و ۲۰۰۰ مختصات پیشنهادی. مشکل اینجاست که شبکه‌های عصبی در لایه‌های نهایی (Fully Connected) نیاز به ورودی با اندازه کاملاً ثابت دارند، اما این ۲۰۰۰ ناحیه اندازه‌های مختلفی دارند. لایه RoI Pooling (Region of Interest) مختصات هر ناحیه پیشنهادی را روی نقشه ویژگی پیدا می‌کند، آن بخش را بُرش می‌زند و با استفاده از عملیات Max Pooling، تمام آن‌ها را به یک ماتریس با ابعاد کاملاً ثابت و یکسان (مثلاً ۷ در ۷) تبدیل می‌کند. سر تشخیص‌دهنده (Detection Head): این ویژگی‌های هم‌اندازه‌شده، به لایه‌های Fully Connected داده می‌شوند و احتمال تعلق آن ناحیه به کلاس‌های مختلف یا کلاس پس زمینه را محاسبه می‌کند. رگرسیون ۴ عدد پیوسته (مختصات مرکز، عرض و ارتفاع) خروجی می‌دهد تا اگر شیء پیدا شد، کادر را دقیقاً روی لبه‌های شیء تراز و فیت کند.

معماری Faster R-CNN:

این معماری الگوریتم‌های خارجی مثل Selective Search که سرعت کل سیستم پایین می‌آورد را حذف کرد و بجایش تصمیم گرفت خودش یک شبکه عصبی داخلی برای پیدا کردن نواحی بسازد (RPN)

ورودی: تصویر از یک شبکه CNN پایه عبور می‌کند و Feature Map تولید می‌شود. تا اینجا شبیه مدل قبلی است ولی این Feature Map قرار است به دو جا برود.

شبکه Region Proposal Network (RPN): شبکه کوچک Fully Convolutional است که بر روی Feature Map اعمال می‌شود (شبیه sliding window). در هر خانه‌ای که این پنجره متوقف می‌شود، شبکه مجموعه‌ای از کادرهای از قبل تعریف شده به نام Anchor Boxes را در نظر می‌گیرد. شبکه برای هر کدام از این Anchor ها دو چیز را پیش‌بینی می‌کند: Objectness Score که می‌گوید آیا داخل این شیء هست یا فقط پس‌زمینه است و ۴ عدد رگرسیون اولیه که مختصات Anchor را کمی جابه‌جا می‌کند تا شیء را بهتر در بر بگیرد.

فیلتر کردن و RoI Pooling: شبکه RPN هزاران Anchor تولید می‌کند. مدل، آن‌هایی که امتیاز شیء بودنشان بسیار پایین است یا هم‌پوشانی شدیدی دارند (از طریق NMS) را حذف می‌کند. حالا نواحی باقی‌مانده‌ی بسیار باکیفیت، همراه با همان نقشه ویژگی اصلی، وارد لایه RoI Pooling می‌شوند تا دوباره اندازه‌شان ثابت شود.

سر تشخیص‌دهنده (Detection Head): دقیقاً شبیه مدل قبلی

دو مزیت Faster R-CNN نسبت به Fast R-CNN

سرعت بسیار بالاتر: در Fast R-CNN، گلوگاه اصلی سرعت، الگوریتم Selective Search بود که بر CPU اجرا می‌شود. در Faster R-CNN، این الگوریتم حذف شد و شبکه عصبی RPN جایگزین آن شد. شبکه RPN روی GPU اجرا می‌شود و مهم‌تر از آن، محاسبات ویژگی‌ها (Feature Maps) را با شبکه اصلی به اشتراک می‌گذارد (Feature Sharing)؛ یعنی نیازی به پردازش مجدد تصویر برای پیدا کردن کاندیداها نیست.

کیفیت بهتر پیشنهادات: الگوریتم Selective Search در Fast R-CNN یک الگوریتم ثابت پردازش تصویر بود و هیچ قدرت یادگیری نداشت؛ یعنی فرقی نمی‌کرد داریم مدل را برای تشخیص چهره آموزش می‌دهیم یا تشخیص اشکال هندسی، این الگوریتم همیشه به یک شکل کورکورانه کاندیدا تولید می‌کرد. اما در Faster R-CNN با جایگزینی RPN، بخش پیشنهاددهنده تبدیل به یک شبکه عصبی قابل آموزش شد. در نتیجه، RPN یاد می‌گیرد الگوهایش را بر طبق دیتاست مورد نظرمون پیدا کند. علاوه بر این، چون کل سیستم یک شبکه‌ی یکپارچه شد، می‌توان کل مدل را با Backpropagation آموزش داد.

۳.۱.۱ معماری Faster R-CNN

نقش Anchor ها در مدل Faster R-CNN چیست و چگونه ایفای نقش می‌کنند؟

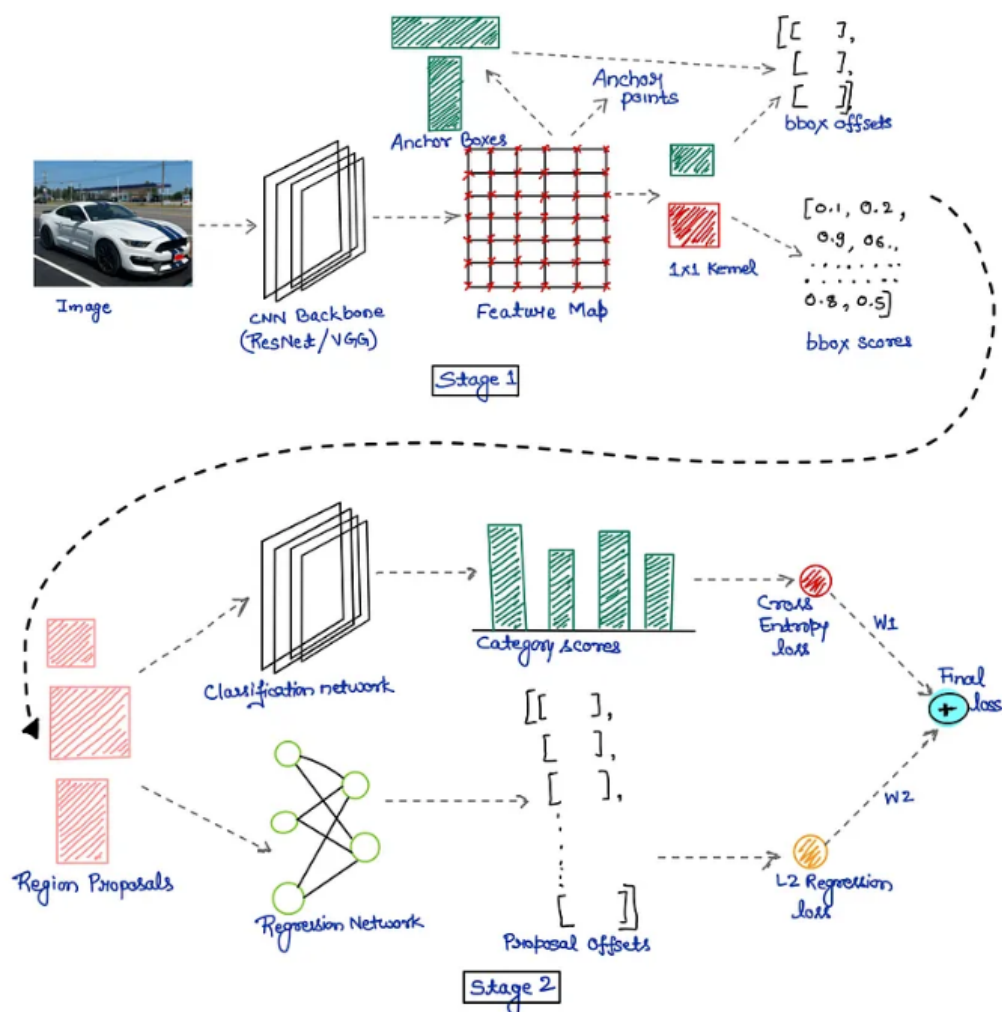
- به جای اینکه شبکه عصبی بخواهد مختصات یک جعبه را از صفر و در فضای خالی پیش‌بینی کند (که یادگیری آن برای شبکه بسیار سخت است)، ما مجموعه‌ای از جعبه‌های از پیش تعریف شده با ابعاد و نسبت‌های مختلف را به عنوان پایه (Proposal های اولیه) در اختیار شبکه قرار می‌دهیم.
- شبکه RPN یک Sliding Window روی تمام خانه‌های Feature Map می‌لغزاند.
- در مرکز هر خانه، k Anchor با شکل‌های مختلف (مثلاً ۳ مقیاس مختلف: کوچک، متوسط، بزرگ $3 \times$ نسبت ابعاد مختلف: ۱:۱، ۱:۲، ۲:۱) قرار می‌گیرد. معمولاً $k = 9$ است.
- شبکه برای هر Anchor، دو خروجی تولید می‌کند:

امتیاز احتمال (Objectness Score): آیا داخل این لنگر پس‌زمینه است یا یک شیء واقعی؟

رگرسیون مختصات (Bounding Box Regression): ۴ عدد Offset شامل جابه‌جایی‌های Δx ، Δy ، Δw و Δh نسبت به همان Anchor اولیه تولید می‌کند تا Anchor را تغییر شکل داده و دقیقاً دور شیء فیت بشوند.

توجه کنید که ما از همون kernel 1x1 داریم استفاده می‌کنیم و این offset و احتمالات در ابتدا معنی ای ندارند، اما به مرور زمان که loss در طول شبکه

backpropagate میشود، چون ما این اعداد را به عنوان offset هامون گرفتیم و جمع میکنیم، شبکه برای اینکه loss کمتر تولید کند مجبور هست چنین معنایی به این اعداد بدهد.



شکل ۲.۱: تصویری معماری Faster R-CNN (از medium.com برداشته شده است).

تکنیک Non-Maximum Suppression (NMS) چیست؟

تکنیک NMS یک الگوریتم Post-processing می‌باشد که وظیفه آن فیلتر کردن و حذف کادرهای پیش‌بینی شده‌ی اضافی و تکراری است.

- ابتدا تمام جعبه‌های پیش‌بینی شده توسط شبکه بر اساس Confidence Score به صورت نزولی مرتب می‌شوند.
- جعبه‌ای که بالاترین امتیاز را دارد به عنوان پیش‌بینی قطعی انتخاب شده و از لیست کاندیداها خارج می‌شود.
- الگوریتم میزان هم‌پوشانی (IoU یا Intersection over Union) این جعبه‌ی قطعی را با تمام جعبه‌های باقی‌مانده در لیست محاسبه می‌کند.
- هر جعبه‌ای که میزان IoU آن با جعبه قطعی، از یک آستانه مشخص (مثلاً 0.7) بیشتر باشد، حذف می‌شود؛ چون این هم‌پوشانی زیاد نشان می‌دهد که

هر دو جعبه دارند به یک شیء واحد اشاره می‌کنند، اما امتیازش کمتر است. این فرآیند برای جعبه بعدی که حالا بالاترین امتیاز را در لیست باقی‌مانده دارد تکرار می‌شود، تا زمانی که لیست خالی شود.

چرا در Faster R-CNN استفاده از تکنیک NMS الزامی است؟

وقتی تصویر وارد شبکه پایه (Backbone) می‌شود و Feature Map تولید می‌گردد، شبکه RPN روی هر پیکسل این نقشه، k anchor قرار می‌دهد. اگر نقشه ویژگی ما ابعادی مثلاً برابر با 50×50 داشته باشد، شبکه RPN در همان نگاه اول $22,500 = 50 \times 50 \times 9$ جعبه روی تصویر می‌اندازد (با فرض $k = 9$). اگر یک شیء در تصویر باشد، دهها anchor در مجاورت هم، بخش‌های مختلف این شیء را پوشش می‌دهند و شبکه RPN به همه‌ی آن‌ها Objectness Score بالایی می‌دهد. اینجا شبکه یعنی کادرهای بسیار شبیه به هم و تو در تو دارد که منطقاً باید فیلتر بشوند.

در دو جا از NMS استفاده میشود، اول، بعد از دهها هزار anchor تولید شده توسط RPN است (توجه کنید که همه اینا رو اگر بدیم، شبکه باید ویژگی‌های دهها هزار کادر را بُرش بزند و پردازش کند و سرعت را خیلی کاهش میدهد). دومین باری که NMS اجرا می‌شود، در انتهای کار و پس از خروجی لایه‌های طبقه‌بند است، با اجرای این از بین این جعبه‌هایی که کلاس یکسانی دارند و به شدت روی هم افتاده‌اند، فقط آن کادری که بالاترین احتمال (Confidence) را دارد نگه می‌دارد.

چرا می‌توان گفت در این معماری Anchorها متراکم (Dense) هستند؟ و این متراکم بودن چه مشکلی ایجاد می‌کند؟

زیرا Anchorها به صورت دستی و سیستماتیک روی تمامی خانه‌های Grid مربوط به Feature Map اعمال می‌شوند یعنی اگر Feature Map ابعاد $H \times W$ داشته باشد و در هر نقطه k anchor قرار دهیم، تعداد کل کاندیداهای تولید شده $H \times W \times k$ خواهد بود که به صدها هزار کاندیدای پوشش‌دهنده کل تصویر می‌رسد. مشکلاتی که این Dense بودن به وجود می‌آورد:

- نتایج تکراری و وابستگی به NMS: این پایپ‌لاین‌ها معمولاً نتایج اضافی و تقریباً مشابهی تولید می‌کنند که یعنی مجبوریم از NMS استفاده کنیم که کند هست.
- در طول آموزش، تعداد بسیار زیادی از کاندیدها به یک شیء واحد اختصاص داده می‌شوند و ما باید دستی یک هیوریستیک‌هایی تعریف کنیم (مثلاً بر اساس شرط IoU) و شبکه به این چیزایی که ما تعریف می‌کنیم بسیار حساس می‌باشد (در Sparse RCNN ولی چون یک به یک هست دیگر نیاز به تعریف هیوریستیک نداریم).

۴.۱.۱ معماری Sparse R-CNN

کلمه Sparse در این مدل دقیقاً به چه چیزی اشاره می‌کند؟

- جعبه‌های پراکنده (Sparse Boxes): به این معناست که یک تعداد بسیار کم و ثابت از جعبه‌های اولیه (مثلاً فقط ۱۰۰ کادر) برای پیش‌بینی تمامی اشیاء موجود در یک تصویر کافی است. این برخلاف روش‌های قبلی است که صدها هزار کاندیدا تولید می‌کردند.
- ویژگی‌های پراکنده (Sparse Features): به این معناست که ویژگی استخراج شده برای هر جعبه، نیازی ندارد که با تمام ویژگی‌های دیگر در عکس interact داشته باشد.

این مدل چه چیزهایی را حذف و چه چیزهایی را اضافه می‌کند؟ (بررسی ۳ تغییر مهم و تاثیر آن‌ها)

- حذف Dense Anchors: این مدل استفاده از شبکه‌هایی که صدها هزار کاندیدای از پیش تعریف شده (Anchor) روی تصویر می‌انداختند را به طور کامل حذف کرده است. با این کار مشکل تخصیص چند به یک (Many-to-one) که در آن چندین کادر به یک شیء نسبت داده می‌شدند برطرف می‌شود (یعنی الگوریتم هیوریستیک IoU نیز حذف می‌شود چون در این مدل یک به یک است).
- اضافه شدن جعبه‌ها و ویژگی‌های پیشنهادی قابل یادگیری: به جای حدس زدن کادرها با RPN، این مدل یک مجموعه کوچک و ثابت (مثلاً ۱۰۰ تایی) از کادرهای ۴-بُعدی و بردارهای ویژگی نهان با ابعاد بالا (مثلاً ۲۵۶) را به عنوان پارامترهای شبکه تعریف می‌کند. این پارامترها در طول آموزش آپدیت می‌شوند. تاثیر آن این است که شبکه به صورت آماری یاد می‌گیرد اشیاء معمولاً کجای تصویر ظاهر می‌شوند (از طریق جعبه‌ها) و ویژگی‌های مانند حالت و شکل اشیاء را نیز کدگذاری می‌کند در طول آموزش. (از طریق بردارهای ویژگی). (یعنی بجای اینکه k تا anchor از قبل تعریف شده داشته باشیم، می‌گذاریم خود شبکه یاد بگیرد و در طول آموزش با توجه به دیتایی که داریم، تغییر بده سایز جعبه‌ها رو و ویژگی‌های proposal هم در طول آزمایش بفهمد).
- اضافه شدن Dynamic Head: در Faster R-CNN ویژگی‌های برش خورده‌ی تمام کادرها وارد یک لایه Fully Connected کاملاً ثابت و یکسان می‌شدند. اما در اینجا، هر ویژگی استخراج شده (RoI Feature) وارد یک head اختصاصی می‌شود که پارامترهای آن توسط همان ویژگی‌های پیشنهادی قابل یادگیری به صورت پویا تولید می‌شوند. این تغییر باعث می‌شود عملیات فیلتر کردن و استخراج ویژگی نهایی به شدت Flexible و دقیق‌تر شود.

این معماری به جای RPN که مربوط به Faster R-CNN هست، چه راهکاری را انتخاب کرده است؟

این مدل برای جایگزینی صدها هزار کاندیدای RPN، از جعبه‌های پیشنهادی قابل یادگیری استفاده می‌کند. به جای اینکه یک شبکه Sliding Window روی تصویر حرکت کند و کادر تولید کند، Sparse R-CNN یک ماتریس با ابعاد $N \times 4$ (مثلاً ۱۰۰ در ۴) تعریف می‌کند که مقادیر آن اعدادی بین ۰ تا ۱ هستند (نشان‌دهنده مختصات مرکز، عرض و ارتفاع نرمال شده). این اعداد در ابتدای کار مقادیر تصادفی دارند، اما در طول آموزش شبکه عصبی، یاد می‌گیرند که بهینه‌ترین نقاط شروع برای پیدا کردن اشیاء در مجموعه داده کجا هستند.

آیا این معماری هنوز نیاز به NMS دارد؟

خیر، در Faster R-CNN دیدیم که NMS برای سرکوب کردن کادرهای اضافی که دور یک شیء واحد می‌افتادند الزامی بود. در اینجا دیگه اون dense بودن را نداریم چون بجای ۱۰۰ هزار کادر تنها ۱۰۰ کادر روی عکس قرار دادیم و احتمال اینکه چندین کادر روی یک شیء تصویر قرار بگیرد بسیار کمتر میشود. Sparse R-CNN از تطبیق دوطرفه (Bipartite Matching) استفاده می‌کند. در طول آموزش، شبکه را مجبور می‌کند که برای هر شیء در تصویر، دقیقاً و فقط یک کادر پیشنهادی را اختصاص دهد و بقیه کادرها را به عنوان پس‌زمینه جریمه کند. با معرفی این تطبیق یک-به-یک، شبکه یاد می‌گیرد که خودش ذاتاً نتایج تکراری تولید نکند؛ بنابراین نیاز به تکنیک دستی NMS نیست.

۲.۱ پیاده‌سازی

۱.۲.۱ آماده‌سازی داده

با توجه به محدودیت‌های پردازشی، داده‌های مصنوعی تولید کرده و مدل Sparse R-CNN را بر روی آن اجرا می‌کنیم.

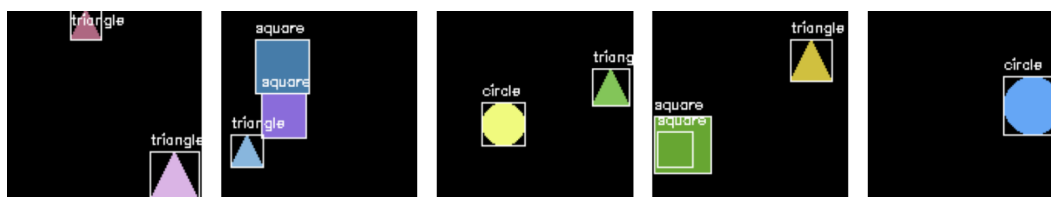
- ۱۰۰۰ تصویر با ابعاد 128x128 پیکسل که مقدار تمامی پیکسل‌های آن‌ها صفر (رنگ مشکی) تنظیم شده است، برای پیش‌زمینه درست کردیم.
- در هر تصویر، به صورت کاملاً تصادفی بین ۱ تا ۳ شکل هندسی رسم می‌شود. این اشکال از میان دایره، مربع و مثلث انتخاب می‌شوند.

- ابعاد هر شکل به صورت تصادفی بین ۲۰ تا ۴۰ پیکسل و رنگ آن‌ها نیز به صورت رندوم تولید می‌شود. همچنین مختصات قرارگیری اشکال به گونه‌ای محاسبه می‌شود که کاملاً درون کادر 128x128 قرار بگیرند و از لبه‌ها بیرون نزنند.

هم‌زمان با رسم هر شکل روی تصویر، اطلاعات مربوط به موقعیت و نوع آن به عنوان برچسب ذخیره می‌شود:

- **جعبه‌های مرزی:** مختصات هر شکل در قالب یک لیست به صورت $[x, y, w, h]$ ذخیره می‌گردد؛ که در آن x و y مختصات گوشه بالا سمت چپ، w عرض و h ارتفاع جعبه مرزی هستند.
- **کلاس‌ها:** برای دایره عدد ۱، برای مربع عدد ۲ و برای مثلث عدد ۳ اختصاص داده شد. (کلاس ۰ نیز به صورت ضمنی برای پس‌زمینه در نظر گرفته شده است).

برای اعتبارسنجی فرآیند تولید داده، ۵ نمونه تصادفی از مجموعه داده ساخته شده، همراه با جعبه‌های مرزی و نام کلاس‌ها استخراج شد که خروجی آن در ادامه قابل مشاهده است:



شکل ۳.۱: پنج نمونه تصادفی از مجموعه داده

۲.۲.۱ تقسیم‌بندی داده‌ها و ساخت dataloader

با استفاده از `transforms.ToTensor()`، تصاویر از آرایه‌های Numpy به تانسورهای پایتورچ تبدیل می‌شوند. این تابع همچنین مقادیر پیکسل‌ها را به صورت خودکار بین ۰ تا ۱ نرمال‌سازی می‌کند.

در مجموعه داده‌ی اولیه، مختصات به صورت $[x, y, w, h]$ و با واحد پیکسل ذخیره شده بودند. برای همخوانی با معماری‌هایی نظیر Sparse R-CNN، این مقادیر ابتدا به فرمت $[x_min, y_min, x_max, y_max]$ تبدیل شدند و سپس با تقسیم بر ابعاد تصویر (۱۲۸)، در بازه $[0, 1]$ نرمال‌سازی گشتند. در مسئله‌ی تشخیص اشیاء، اگرچه تمامی تصاویر ورودی در یک بچ هم‌اندازه هستند (مثلاً 128x128)، اما تعداد اشیای موجود در هر تصویر (و در نتیجه ابعاد برچسب‌های آن) متغیر است (مثلاً یک تصویر دارای ۱ شکل و دیگری دارای ۳ شکل است).

تابع پیش‌فرض دیتالودر سعی می‌کند تمام اجزای یک بچ را درون یک تانسور واحد Stack کند که در مواجهه با برچسب‌های با طول متغیر با خطا مواجه می‌شود. برای رفع این مشکل، تابع `collate` سفارشی نوشته شد تا:

- تصاویر: به صورت یکپارچه در یک تانسور ۴-بعدی Stack شوند.
- برچسب‌ها: به جای تبدیل شدن به تانسور یکپارچه، در قالب یک لیست از دیکشنری‌ها حفظ شوند تا ساختار متغیر خود را از دست ندهند.

از مجموع ۱۰۰۰ داده‌ی تولیدشده، ۹۰۰ داده برای آموزش (`train_split=900`) و ۱۰۰ داده برای تست در نظر گرفته شد. سپس دیتالودرهای آموزش و تست با اندازه بچ ۱۶ ساخته شدند.

برای اطمینان از صحت عملکرد دیتالودر، یک بچ از داده‌ها استخراج و ساختار آن بررسی شد که خروجی آن به شرح زیر است:

```
Images Tensor Shape: torch.Size([16, 3, 128, 128])
```

```
Images Data Type: torch.float32
```

```
Length of Targets List: 16
```

```
First Target in the Batch:
```

```
Boxes Tensor:
```

```
tensor([[0.2891, 0.0312, 0.5000, 0.2422],
        [0.5312, 0.2812, 0.7422, 0.4922],
        [0.5625, 0.4922, 0.8281, 0.7578]])
```

```
Labels Tensor:
```

```
tensor([1, 2, 1])
```

تانسور خروجی به درستی نشان‌دهنده‌ی یک بچ با ۱۶ تصویر، ۳ کانال رنگی (RGB) و ابعاد پیکسلی 128x128 است. نوع داده نیز به درستی به اعشاری (float32) تبدیل شده است.

طول لیست برچسب‌ها ۱۶ نشان می‌دهد که تابع collate به درستی عمل کرده و برای هر تصویر یک دیکشنری مجزا در لیست قرار داده است. در نمونه‌ی نمایش داده‌شده، تصویر اول شامل ۳ شکل هندسی است. مختصات جعبه‌های مرزی همگی مقادیر اعشاری بین ۰ و ۱ هستند (نرمال‌سازی موفق) و برچسب‌های متناظر آن‌ها ([1, 2, 1]) نشان‌دهنده‌ی حضور دو دایره و یک مربع در این تصویر خاص است.

۳.۲.۱ پیاده‌سازی معماری Sparse R-CNN

برخلاف معماری‌های کلاسیک مانند Faster R-CNN که برای تولید نواحی پیشنهادی به یک شبکه مجزا (RPN) و هزاران Anchor متکی هستند، این معماری از تعداد محدودی پیشنهاد (Proposal) قابل یادگیری استفاده می‌کند که به صورت سراسری روی کل داده‌ها بهینه می‌شوند. این شبکه از سه بخش اصلی تشکیل شده است:

۱. معماری Backbone:

برای استخراج ویژگی‌های تصویری، یک شبکه عصبی کانولوشنی ساده شامل ۳ بلوک طراحی شده است.

- ورودی: تانسوری با ابعاد [Batch, 3, 128, 128].
- بلوک اول: استفاده از لایه کانولوشن با stride برابر ۲ باعث افزایش عمق کانال‌ها به ۳۲ و کاهش ابعاد مکانی به نصف می‌شود (خروجی: [Batch, 32, 64, 64]).
- بلوک دوم: افزایش کانال‌ها به ۶۴ و کاهش مجدد ابعاد (خروجی: [Batch, 64, 32, 32]).
- بلوک سوم: افزایش کانال‌ها به ۱۲۸ و کاهش ابعاد برای رسیدن به نقشه ویژگی نهایی (خروجی: [Batch, 128, 16, 16]).

پس از هر لایه کانولوشن، از BatchNorm2d استفاده شده است تا خروجی‌ها نرمال شوند؛ این کار پایداری آموزش را به شکل قابل توجهی افزایش می‌دهد و به دنبال آن از تابع فعال‌ساز ReLU استفاده شده است.

۲. معماری Dynamic Head:

این ماژول وظیفه دارد ویژگی‌های استخراج‌شده از تصویر (RoI Features) را با ویژگی‌های پیشنهادی (Proposal Features) ترکیب کرده و پیش‌بینی نهایی را انجام دهد. فرآیند به این شکل است:

- ابتدا ویژگی‌های پیشنهادی مسطح‌شده و از یک لایه خطی عبور می‌کنند تا به یک بردار با ابعاد بالا تصویر شوند. این خروجی سپس به دو بخش param1 و param2 تقسیم می‌شود. این دو پارامتر در واقع نقش وزن‌های سفارشی‌سازی‌شده را برای هر Proposal ایفا می‌کنند.
- به‌جای استفاده از کانولوشن‌های دوبعدی استاندارد، شبکه از ضرب ماتریسی دسته‌ای (torch.bmm) استفاده می‌کند. در این حالت، هر ویژگی RoI منحصراً با پارامترهای تولیدشده توسط ویژگی پیشنهادی متناظر خودش تعامل می‌کند. پس از اعمال LayerNorm و ReLU، خروجی‌ها مسطح شده و از یک لایه خطی عبور می‌کنند تا refined_features (ویژگی‌های اصلاح‌شده) به دست آید.
- سر شبکه با استفاده از یک لایه خطی (cls_layer)، احتمال کلاس‌ها را پیش‌بینی می‌کند و با استفاده از یک MLP سه‌لایه به همراه ReLU، تغییرات مورد نیاز برای اصلاح مختصات جعبه‌ها (box_deltas) را محاسبه می‌کند.
- ویژگی‌های اصلاح‌شده (refined_features) در معماری تکرارشونده، به عنوان ویژگی‌های پیشنهادی برای مرحله‌ی بعدی استفاده خواهند شد.

۳. معماری Sparse R-CNN:

- حذف RPN و تولید پیشنهادات یادگیرنده: شبکه تعداد ثابتی جعبه پیشنهادی (در اینجا num_proposals=20 با مختصات $4 \times N$) ایجاد می‌کند. این مختصات با مقادیر تصادفی در بازه [0, 1] مقداردهی اولیه شده و درون nn.Parameter قرار می‌گیرند تا در طول Back-propagation همراه با شبکه آموزش ببینند. این پارامترها در واقع آماری از کل دیتاست هستند که یاد می‌گیرند اشیاء معمولاً کجای تصویر قرار دارند.
- فرایند Iterative Refinement: شبکه در یک حلقه با ۳ مرحله (Stage) پیش‌بینی‌ها را به تدریج بهبود می‌بخشد:

– با استفاده از مختصات فعلی جعبه‌ها و عملیات torchvision.ops.roi_align، ویژگی‌های مکانی با اندازه ثابت (7×7) از نقشه ویژگی Backbone استخراج می‌شوند. (مقیاس مکانی یا spatial_scale برابر با $1/16$ تنظیم شده است، زیرا ابعاد تصویر از ۱۲۸ به ۱۶ کاهش یافته است).

– ویژگی‌های RoI و ویژگی‌های پیشنهادی به DynamicHead داده می‌شوند تا cls_logits، box_deltas و prop_features جدید تولید شوند.

– مقادیر box_deltas به جعبه‌های فعلی اضافه می‌شوند تا مختصات آن‌ها اصلاح گردد.

– برای جلوگیری از خطای شبکه و نامعتبر شدن جعبه‌ها، بلافاصله پس از آپدیت، مختصات جعبه‌ها توسط torch.clamp در بازه [0.0, 1.0] محدود می‌شوند. علاوه بر این، با استفاده از توابع min و max و یک مقدار بسیار کوچک $(\epsilon = 1e - 3)$ ، یک محدودیت هندسی سفت و سخت اعمال می‌شود تا همواره شرط $x_1 < x_2$ و $y_1 < y_2$ برقرار بماند.

۴.۲.۱ تعریف تابع هزینه و تطبیق دوطرفه

۱. الگوریتم تطبیق مجارستانی (Hungarian Matching)

به‌جای تکیه بر قوانین انتساب برچسب چند-به-یک (مانند تخصیص چندین Anchor متراکم به یک شیء واقعی بر اساس حد آستانه IoU که در NMS استفاده می‌شود)، این تطبیق‌دهنده یک رابطه و انتساب یک-به-یک و دقیق میان پیش‌بینی‌های شبکه و اشیای واقعی در تصویر برقرار می‌کند.

برای یافتن بهترین تطبیق، ابتدا یک ماتریس هزینه محاسبه می‌شود که خود از سه بخش تشکیل شده است:

- **هزینه طبقه‌بندی (Classification Cost):** خروجی‌های خام شبکه (Logits) از یک تابع فعال‌ساز sigmoid عبور می‌کنند تا احتمالات کلاس استخراج شوند. هزینه بر اساس احتمال منفی کلاس واقعی محاسبه می‌شود (هرچه احتمال پیش‌بینی کلاس صحیح بالاتر باشد، هزینه کمتر است).
- **هزینه فاصله (L1 Distance Cost):** محاسبه فواصل مطلق بین مختصات مرکز، عرض و ارتفاع جعبه‌های پیش‌بینی شده و جعبه‌های واقعی. اگرچه خطای L1 برای تنظیمات خام مختصات مؤثر است، اما نسبت به مقیاس مستقل نیست (Scale-variant)؛ یعنی یک خطای کوچک پیکسلی در یک جعبه بزرگ، همان‌قدر جریمه می‌شود که در یک جعبه بسیار کوچک جریمه می‌گردد.
- **هزینه اشتراک تعمیم‌یافته (Generalized IoU Cost):** محاسبه GIoU منفی. برخلاف فاصله L1 استاندارد، معیار GIoU نسبت به مقیاس مستقل است (scale-invariant) و جعبه‌هایی که هم‌پوشانی خوبی با هدف ندارند را جریمه می‌کند.

در نهایت، این هزینه‌های منفرد با ضرایب مشخص خود ضرب شده و جمع می‌شوند تا هزینه کل به دست آید. تابع `scipy.optimize.linear_sum_assignment` الگوریتم مجارستانی را برای حل این مسئله انتساب خطی اعمال می‌کند. خروجی این تابع، ترکیبی بهینه از جفت‌های پیش‌بینی-هدف است که کمترین هزینه تطبیق کلی را برای آن تصویر به همراه دارد. این جفت‌های استخراج‌شده، متعاقباً توسط ماژول `SparseRCNNCriterion` برای محاسبه میزان خطای واقعی جهت پس‌انتشار (Backpropagation) استفاده می‌شوند.

۲. توابع هزینه (Loss Functions)

خطای کل برای جفت‌های تطبیق‌داده‌شده، یک ترکیب خطی وزن‌دار از خطای طبقه‌بندی و دو خطای رگرسیون جعبه‌های مرزی است:

$$\mathcal{L} = \lambda_{cls} \mathcal{L}_{cls} + \lambda_{L1} \mathcal{L}_{L1} + \lambda_{giou} \mathcal{L}_{giou}$$

که در آن ضرایب استاندارد مطابق مقاله به صورت $\lambda_{cls} = 2.0$ ، $\lambda_{L1} = 5.0$ و $\lambda_{giou} = 2.0$ تنظیم شده‌اند.

الف) خطای طبقه‌بندی (Classification Loss - Focal Loss)

از آنجایی که اکثریت قاطع پیشنهاد‌های اولیه با ”پس‌زمینه” (مکان‌های بدون شیء) مطابقت پیدا می‌کنند، شبکه با یک عدم تعادل شدید کلاس‌ها (Class Imbalance) مواجه است. برای جلوگیری از تسلط کلاس پس‌زمینه بر گرادین‌های خطا، به جای آنتروپی متقاطع استاندارد، از Focal Loss استفاده شده است:

$$FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

- متغیر p_t : احتمال تخمینی مدل برای کلاس هدف (که با فعال‌ساز sigmoid محاسبه می‌شود).
- متغیر γ (پارامتر تمرکز): برابر با ۰.۲ تنظیم شده است. این پارامتر جریمه‌ی نمونه‌های آسان (جایی که p_t به ۱ نزدیک است) را به آرامی کاهش می‌دهد و مدل را مجبور می‌کند تا روی نمونه‌های سخت و اشتباه طبقه‌بندی‌شده تمرکز کند.
- متغیر α_t (عامل تعادل): برابر با ۲۵.۰ تنظیم شده است تا اهمیت نمونه‌های مثبت و منفی را متعادل کند.

ازی: درکد، برای پایداری عددی و جلوگیری از تولید مقادیر NaN، به جای پیاده‌سازی دستی لگاریتم، از تابع داخلی پایتورچ یعنی `F.binary_cross_entropy_with_logits` استفاده شده است).

ب) خطای رگرسیون جعبه‌های مرزی (loss_boxes)

مدل فقط پیشنهادهایی را ارزیابی می‌کند که توسط الگوریتم مجارستانی با موفقیت به یک شیء واقعی تطبیق داده شده باشند. این خطا خود شامل دو بخش است:

- **خطای L1:** این خطا فاصله مطلق پیکسلی بین مختصات جعبه پیش‌بینی شده و هدف واقعی را محاسبه می‌کند.

$$\mathcal{L}_{L1} = \frac{1}{N} \sum_{i=1}^N |b_i^{pred} - b_i^{gt}|$$

- **خطای GIoU:** برای حل مشکل حساسیت به مقیاس در خطای L1، از GIoU استفاده می‌شود.

$$\mathcal{L}_{giou} = 1 - \left(IoU - \frac{|C \setminus (A \cup B)|}{|C|} \right)$$

در این فرمول، A و B جعبه‌ی پیش‌بینی شده و واقعی هستند و C کوچک‌ترین فضای محدب دربرگیرنده (Convex Hull) است (کوچک‌ترین جعبه ممکن که هر دو ناحیه A و B را کاملاً پوشش دهد). اگر دو جعبه هیچ تقاطعی نداشته باشند ($IoU = 0$)، عبارت کسری موجود در فرمول به عنوان جریمه‌ای عمل می‌کند که جعبه پیش‌بینی شده را در طول آموزش به سمت هدف هل می‌دهد.

۳. روند محاسبه در طول Forward Pass:

در طول Forward Pass، کلاس SparseRCNNCriterion پیش‌بینی‌های مربوط به یک مرحله (Stage) خاص از حلقه تکرارشونده شبکه را دریافت کرده و به شکل زیر عمل می‌کند:

- ابتدا self.matcher را فراخوانی می‌کند تا جایگشت‌های بهینه (indices) که پیش‌بینی‌ها را به اهداف واقعی متصل می‌کنند، بیابد.
- سپس loss_labels و loss_boxes را منحصراً با استفاده از همین شاخص‌های تطبیق داده‌شده محاسبه می‌کند.

۵.۲.۱ آموزش مدل

در این بخش، مدل پیاده‌سازی شده با استفاده از داده‌های تولیدی آموزش داده می‌شود. همچنین در پایان هر اپیاک، ارزیابی مدل روی داده‌های تست جهت محاسبه معیار mAP (Mean Average Precision) انجام شده است.

- آموزش برای ۵۰ اپیاک انجام شد تا از همگرایی کامل مدل اطمینان حاصل شود.
- برای آموزش این معماری از ترکیب بهینه‌ساز Adam (با نرخ یادگیری اولیه 5×10^{-4} و weight_decay با مقدار 1×10^{-4}) و زمان‌بند CosineAnnealingLR استفاده شد. بهینه‌ساز Adam با محاسبه گشتاور اول و دوم گرادیان‌ها، یک نرخ یادگیری تطبیقی و اختصاصی برای تک‌تک پارامترهای شبکه تنظیم می‌کند. در کنار آن، زمان‌بند کسینوسی، نرخ یادگیری پایه را در طول ۵۰ اپیاک به نرمی از 5×10^{-4} به 3×10^{-4} کاهش می‌دهد. این رویکرد ترکیبی تضمین می‌کند که شبکه در اپیاک‌های اولیه با سرعت بالا الگوها را بیاموزد و در اپیاک‌های انتهایی با کاهش نرخ پایه، همگرایی دقیق‌تری به سمت مینیمم سراسری تابع هزینه داشته باشد و از نوسانات مخرب جلوگیری شود.

در مسئله تشخیص اشیاء، ارزیابی مدل صرفاً با بررسی صحت کلاس پیش‌بینی شده انجام نمی‌شود، بلکه دقت مکان‌یابی (مختصات جعبه مرزی) نیز اهمیت بالایی دارد. معیار mAP استاندارد برای این ارزیابی است:

- معیار **IoU (Intersection over Union)**: میزان اشتراک مساحت جعبه پیش‌بینی‌شده با جعبه واقعی تقسیم بر مساحت اجتماع آن‌هاست. اگر IoU از یک حد آستانه بیشتر باشد، تشخیص به عنوان True Positive در نظر گرفته می‌شود.
- معیار **AP (Average Precision)**: سطح زیر نمودار Precision-Recall برای یک کلاس خاص است.
- معیار **mAP**: میانگین نمرات AP برای تمامی کلاس‌ها (دایره، مربع و مثلث) است.

در این پیاده‌سازی از $\text{mAP (IoU=0.50:0.95)}$ استفاده شده است. این یعنی به ازای $\text{IoU}=0.50$ تا 0.95 (با گام های 0.05 یعنی 10 تا) ارزیابی شده و بعد میانگین گرفته شده است.

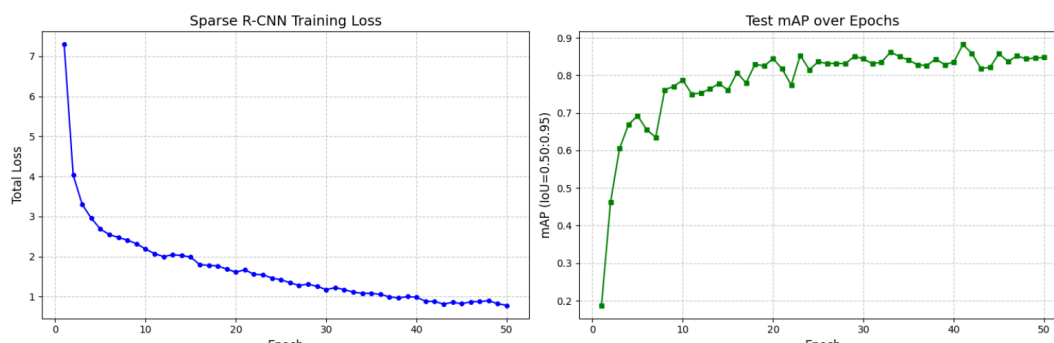
در هر تکرار آموزش، پس از صفر کردن گرادیان‌ها، پیش‌بینی‌های مدل برای تمام مراحل سر تکرارشونده استخراج می‌شوند. تابع هزینه برای تمام مراحل محاسبه می‌شود. یعنی خروجی Stage 1، Stage 2 و Stage 3 تک‌تک با Ground-Truth تطبیق داده شده و خطایشان محاسبه می‌شود و در نهایت این خطاها با هم جمع می‌گردند ($\text{total_loss} += \text{loss_val}$). این تکنیک باعث می‌شود که حتی لایه‌های میانی نیز سیگنال خطای مستقیم دریافت کنند و ویژگی‌های بهتری استخراج نمایند.

همچنین پیش از به‌روزرسانی وزن‌ها (optimizer.step)، از Gradient Clipping با سقف $\text{max_norm}=1.0$ استفاده شد تا از انفجار گرادیان‌ها جلوگیری شود.

محاسبه mAP روی داده های تست:

- تنها از پیش‌بینی‌های آخرین مرحله (Stage 3) استفاده می‌شود زیرا این خروجی‌ها دقیق‌ترین پیش‌بینی‌های مدل هستند.
- با اعمال تابع sigmoid روی خروجی‌های طبقه‌بند، امتیاز هر کلاس به دست آمده و کلاسی که بیشترین احتمال را دارد انتخاب می‌شود.
- پیش‌بینی‌هایی که برچسب ۰ (پس‌زمینه) دریافت کرده‌اند، فیلتر می‌شوند تا فقط کلاس‌های ۱، ۲ و ۳ (دایره، مربع، مثلث) باقی بمانند.
- مختصات جعبه‌ها که قبلاً در بازه $[0, 1]$ نرمال شده بودند، دوباره در عدد 128 ضرب می‌شوند تا به مقیاس پیکسلی تصویر (128×128) بازگردند.
- سپس این مقادیر به تابع MeanAveragePrecision پاس داده می‌شوند.

کد نمودار مربوط به افت خطای آموزش و افزایش دقت mAP را رسم کرده است که در تصویر زیر قابل مشاهده است:



شکل ۴.۱: نمودار آموزش و تست مدل.

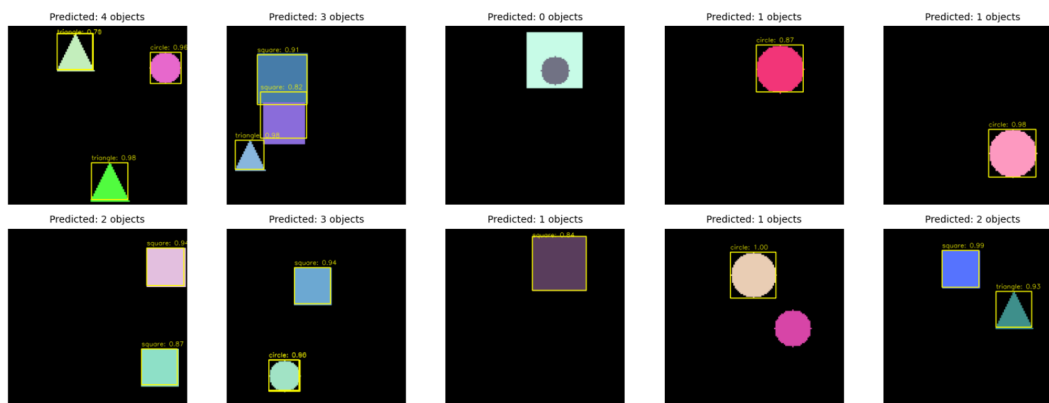
تحلیل:

- مدل در ۵ تا ۱۰ ایپاک ابتدایی افت بسیار شدیدی در خطا دارد که نشان‌دهنده یادگیری سریع الگوهای اولیه (مانند جایگاه تقریبی اشکال و تمایز پس‌زمینه از پیش‌زمینه) است. پس از آن، خطا با یک شیب ملایم و پایدار به سمت مقادیر پایین‌تر حرکت می‌کند که نشان‌دهنده اثر بخشی Adam و Gradient Clipping در پایداری آموزش است.
- معیار mAP (با آستانه تقاطع 0.50:0.95) با سرعت خوبی افزایش یافته و در نهایت تقریباً ۸۵ درصد می‌شود یعنی شبکه توانسته جعبه‌ها را با دقت بسیار بالایی روی اشکال هندسی منطبق کند. افت و خیزهای جزئی در نمودار mAP نیز طبیعی بوده و به دلیل تنوع تصادفی در بچ‌های تست است.

۶.۲.۱ ارزیابی نهایی و تحلیل بصری

در این بخش، خروجی‌های نهایی مدلی که آموزش دیده است، روی داده‌های تست ارزیابی شده و نتایج به صورت بصری نمایش داده می‌شوند. تابع `evaluate_and_visualize` برای محاسبه دقت نهایی روی کل داده‌های تست و رسم پیش‌بینی‌های مدل روی ۱۰ نمونه تصادفی طراحی شده است. روند کار در این تابع مشابه روند ارزیابی در زمان آموزش (توضیح داده شده در بخش قبل) است: خروجی‌های آخرین لایه شبکه (Stage 3) استخراج شده، کلاس‌های پس‌زمینه (لیل ۰) حذف می‌شوند و مختصات نرمال‌شده به پیکسل‌های واقعی (128x128) بازگردانده می‌شوند. برای نمایش خواناتر در تصویر، از یک ضریب مقیاس دهی (`scale_factor=4`) استفاده شده است تا وضوح خروجی‌های بصری با کتابخانه OpenCV افزایش یابد. در نهایت آستانه اطمینان مدل (`score_threshold`) روی 0.3 تنظیم شده است تا تنها پیش‌بینی‌های مطمئن رسم شوند. نتیجه mAP نهایی محاسبه شده روی داده‌های تست برابر با 0.8476 شد.

تصویر زیر خروجی مدل را روی ۱۰ تصویر از مجموعه داده‌ی آزمون نشان می‌دهد:



شکل ۵.۱: پیش‌بینی‌های مدل روی ۱۰ نمونه از داده‌های تست.

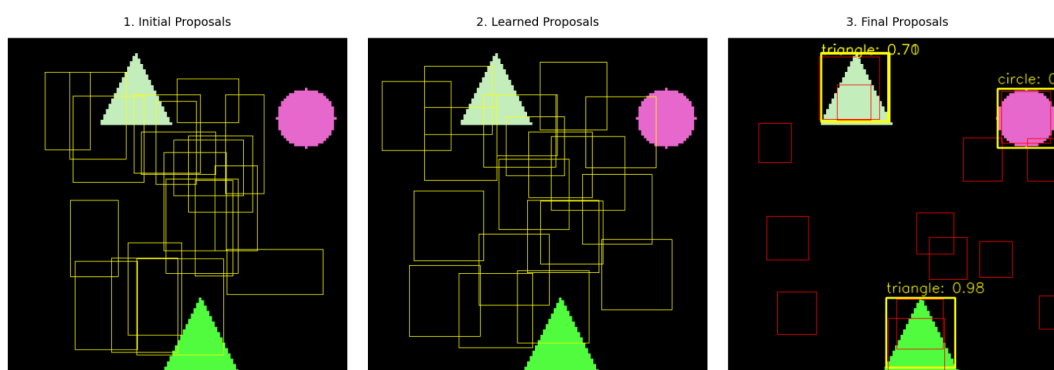
تحلیل:

- شبکه با موفقیت توانسته است اکثر اشکال هندسی را در ابعاد، رنگ‌ها و موقعیت‌های مختلف شناسایی کند.
- دقت دسته‌بندی: برچسب کلاس‌ها به درستی تخصیص داده شده‌اند (مثلاً در تصاویر حاوی مثلث و دایره، شبکه به درستی تفاوت ویژگی‌های آن‌ها را متوجه شده است).
- دقت جعبه‌های مرزی: جعبه‌های رسم شده به صورت کاملاً مماس بر اشکال قرار گرفته‌اند.
- قدرت جداسازی: در تصاویری که اشکال روی هم افتاده‌اند (Overlap)، مدل به درستی دو جعبه مجزا رسم کرده است.

• **نقطه ضعف:** در تصویر سوم از سطر اول، یک دایره تقریباً درون یک مربع قرار گرفته است، اما شبکه هیچ‌کدام را تشخیص نداده و پیش‌بینی صفر شیء داشته است. Backbone طراحی شده در این پروژه، ابعاد تصویر ورودی (128x128) را به یک نقشه ویژگی با ابعاد 16x16 کاهش می‌دهد. این یعنی هر پیکسل در نقشه ویژگی، نماینده‌ی یک ناحیه 8x8 در تصویر اصلی است. وقتی دایره کاملاً درون مربع قرار دارد، ماژول RoI Align هنگام استخراج ویژگی‌های مربع، ناگزیر تمام ویژگی‌های دایره را نیز درون خود بلعیده و استخراج می‌کند. به دلیل رزولوشن بسیار پایین نقشه ویژگی، شبکه نمی‌تواند مرز ظریف و تفاوت سیگنال بین شیء داخلی و شیء بیرونی را درک کند و این دو از نگاه شبکه به یک بافت واحد تبدیل می‌شوند. در معماری Sparse R-CNN برای حذف پست‌پردازش NMS، پیشنهادها در لایه‌های پنهان با یکدیگر تعامل می‌کنند تا از پیش‌بینی‌های تکراری برای یک شیء جلوگیری کنند. در موردی که یک شیء کوچک درون شیء بزرگتر قرار دارد، ویژگی‌های مکانی آن‌ها به شدت هم‌پوشانی دارد. در نتیجه، شبکه ممکن است این دو شیء متفاوت را به عنوان یک شیء واحد که دو بار توسط دو پیشنهاد مختلف انتخاب شده اشتباه بگیرد. در این حالت مکانیزم داخلی شبکه سعی می‌کند با سرکوب کردن، یکی از آن‌ها را حذف کند، اما چون به دلیل تداخل ویژگی‌ها هیچ‌کدام از پیشنهادها نتوانسته‌اند جعبه‌ای دقیق و بی‌نقص تولید کنند، Confidence Score هر دو شکسته شده و به زیر آستانه 0.3 سقوط می‌کند. (کلا Backbone رو پیچیده تر میکردیم یا num_stages بیشتر میکردیم احتمال داشت این را تشخیص بدهد ولی برای چیزی که طراحی کردیم، این مثال پیشرفته تر از طراحی ساده ماست.)

۲. تحلیل بصری تکامل Proposal ها (Evolution of Proposals):

یکی از اهداف اصلی صورت پروژه، بررسی تفاوت جعبه‌های پیشنهادی قبل و بعد از آموزش است. برای این منظور، تابع visualize_proposal_evolution نوشته شد تا وضعیت ۲۰ جعبه پیشنهادی را در ۳ مرحله کلیدی روی یک تصویر واحد نشان دهد.



شکل ۶.۱: روند تکامل ۲۰ جعبه‌ی پیشنهادی

تحلیل:

• مرحله اول (Initial Proposals):

در ابتدای کار، پیش از شروع فرآیند آموزش، مختصات ۲۰ جعبه پیشنهادی به صورت کاملاً تصادفی (با تابع torch.rand) مقاداردهی اولیه شدند. همان‌طور که در تصویر سمت چپ مشخص است، این جعبه‌ها هیچ نظم خاصی ندارند، به صورت درهم‌ریخته در کادر پراکنده شده‌اند و ابعاد آن‌ها نامتناسب است. این وضعیت شبکه پیش از دیدن هرگونه داده‌ای است.

• مرحله دوم (Learned Proposals):

این مرحله وضعیت پارامترهای self.proposal_boxes را پس از ۵۰ اپیاک آموزش (اما قبل از ورود به حلقه‌ی تکرارشونده برای این عکس خاص) نشان می‌دهد. این جعبه‌ها دیگر کاملاً تصادفی نیستند، بلکه در طول آموزش بهینه شده‌اند تا یاد بگیرند اشیاء معمولاً در کجای تصاویر این دیتاست توزیع شده‌اند. با مشاهده‌ی تصویر وسط می‌بینیم که جعبه‌ها به صورت یکنواخت‌تر و با ابعاد منطقی‌تری (مثلاً در مرحله اول یک مستطیل باریک می‌بینیم و

هیچ کدوم از شکلامون در هیچ عکسی بدیهتا درونش قرار نمیگیرد ولی در این مرحله ابعاد جعبه‌ها منطقی تر است. در سراسر تصویر پخش شده‌اند تا به عنوان یک حدس اولیه خوب و سراسری عمل کنند.

• مرحله سوم (Final Proposals):

این تصویر خروجی نهایی شبکه را (پس از عبور ویژگی‌های استخراج شده و ویژگی‌های پیشنهادی از حلقه‌ی سه مرحله‌ای Dynamic Head) نشان می‌دهد. در اینجا می‌بینیم که چگونه مدل با اضافه کردن خروجی لایه رگرسیون (box_deltas) به مختصات جعبه‌های اولیه (Learned Proposals)، آن‌ها را به سمت اشیای واقعی در این تصویر خاص هل داده است. شبکه یاد گرفته است که اکثر پیشنهادهای بی‌استفاده (جعبه‌های قرمز رنگ) را به عنوان کلاس "پس‌زمینه" پیش‌بینی کند و امتیاز بسیار پایینی به آن‌ها بدهد، در نتیجه این جعبه‌ها فیلتر شده و نادیده گرفته می‌شوند. در مقابل، تنها ۳ جعبه (جعبه‌های زرد رنگ ضخیم) توانسته‌اند با اطمینان بالا، خود را با ۳ شکل موجود در تصویر مطابقت دهند یعنی بدون نیاز به الگوریتم NMS، پیشنهادهای اضافه را خاموش می‌کند.